

MỤC LỤC

1. Giới thiệu hệ thống robot mô phỏng

- 1.1 Robot omni-directional
- 1.2 Giới thiệu về bản đồ mô phỏng

2. Các topic ROS2 cần chú ý trong hệ thống

- 2.1 /map
- 2.2 /odom
- 2.3 /amcl_pose
- 2.4 /scan_front_raw
- 2.5 /scan_rear_raw
- 2.6 /tf
- 2.7 /mobile_base_controller/reference

3. Các server trong hệ thống launch Nav2

- 3.1 Map Server (nav2_map_server)
- 3.2 AMCL (nav2_amcl)
- 3.3 Planner Server (nav2_planner)
- 3.4 Controller Server (nav2_controller)
- 3.5 Behavior Server / Recoveries Server (nav2_behaviors)
- 3.6 BT Navigator (nav2_bt_navigator)
- 3.7 Lifecycle Manager

4. Thuật toán tối ưu đường đi

- 4.1 Ý tưởng chính
- 4.2 Cách tính chi phí
- 4.3 Thuật toán tối ưu
- 4.4 Điểm đặc biệt
- 4.5 Kết luận

1.1 Robot omni-directional



Hình 1.1: Robot omni được sử dụng để mô phỏng trong gazebo

Robot sử dụng trong mô phỏng là một robot di động omni-directional với cấu trúc 4 bánh, cho phép di chuyển linh hoạt theo nhiều hướng khác nhau mà không cần thay đổi hướng thân robot. Hình 1.1 minh họa mô hình robot trong môi trường mô phỏng Gazebo.

Về mặt cơ khí, robot có dạng hình hộp chữ nhật với kích thước tương đối gọn, phù hợp di chuyển trong môi trường bệnh viện. Theo mô tả trong file URDF, thân robot (`base_link`) được biểu diễn bằng các khối hộp và chạm với kích thước xấp xỉ 0.7×0.5 m, đảm bảo bao phủ toàn bộ phần thân và phục vụ cho việc tính toán va chạm trong quá trình điều hướng.

Robot được trang bị bốn bánh xe đặt tại bốn góc:

- `wheel_front_right`
- `wheel_front_left`
- `wheel_rear_right`
- `wheel_rear_left`

Mỗi bánh xe được điều khiển độc lập thông qua các khớp dạng quay liên tục (continuous joint) và sử dụng điều khiển vận tốc. Hệ thống điều khiển được cấu hình thông qua

`ros2_control`, cho phép gửi lệnh vận tốc đến từng bánh xe riêng biệt. Điều này giúp robot có thể thực hiện các chuyển động:

- Tiến/lùi
- Di chuyển ngang (trái/phải)
- Xoay tại chỗ

Ngoài hệ thống truyền động, robot còn được trang bị các cảm biến quan trọng phục vụ cho định vị và xây dựng bản đồ:

- Hai cảm biến LiDAR đặt ở phía trước và phía sau (`base_front_laser`, `base_rear_laser`) với góc quét rộng ($\sim 270^\circ$), giúp mở rộng vùng quan sát môi trường.
- Một cảm biến IMU (`base_imu`) hỗ trợ xác định hướng và gia tốc của robot.

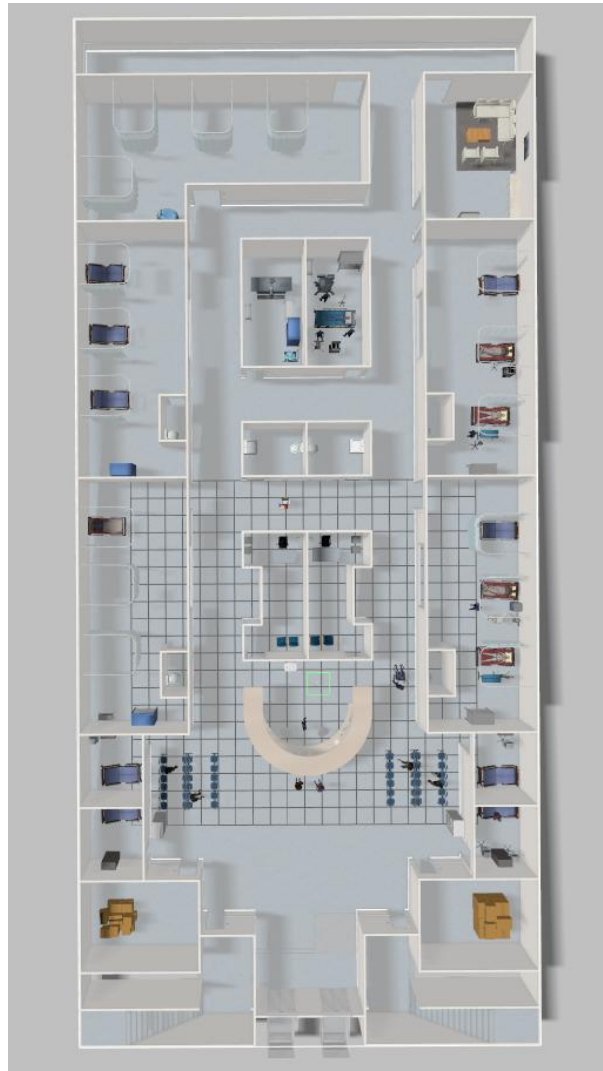
Các cảm biến này được gắn cố định trên thân robot và xuất dữ liệu qua các topic trong hệ thống ROS2, phục vụ cho các thuật toán SLAM và navigation.

Nhờ sự kết hợp giữa hệ truyền động omni-directional và hệ thống cảm biến, robot có khả năng di chuyển linh hoạt và nhận biết môi trường hiệu quả, phù hợp cho các ứng dụng trong không gian phức tạp như bệnh viện.

1.2 Giới thiệu về bản đồ mô phỏng

Môi trường mô phỏng bệnh viện:

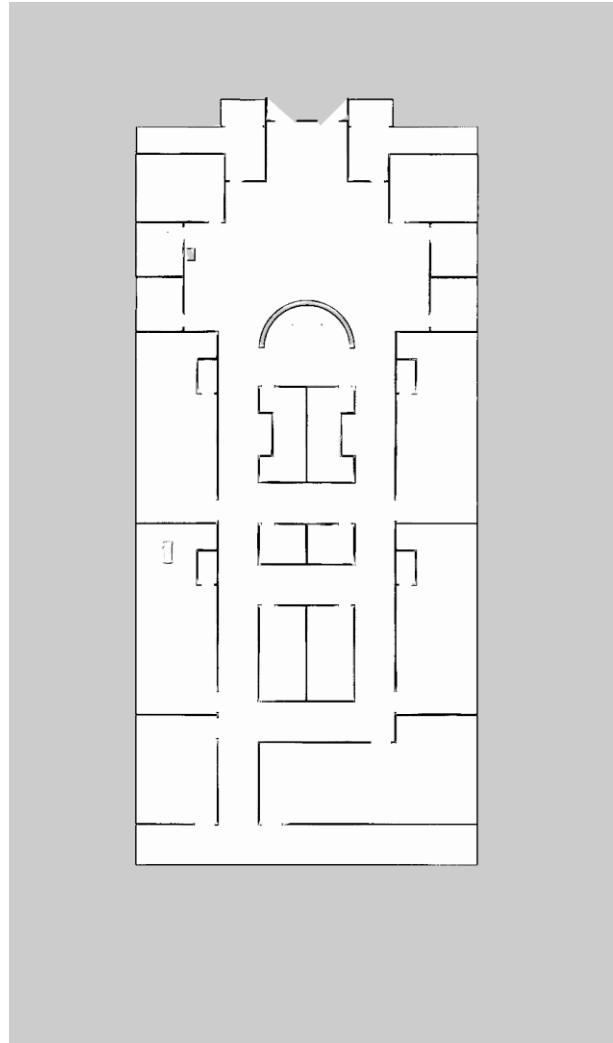
Ảnh mô phỏng thể hiện không gian một tầng bệnh viện được xây dựng trong Gazebo, bao gồm các khu vực như hành lang, phòng bệnh, khu lễ tân và khu chờ. Trong môi trường có nhiều vật thể như giường bệnh, ghế, thiết bị y tế... đóng vai trò là chướng ngại vật. Mô hình này giúp tái hiện điều kiện thực tế để kiểm tra khả năng di chuyển, tránh vật cản và hoạt động tổng thể của robot.



Bản đồ 2D (.pgm):

Ảnh bản đồ .pgm được tạo ra từ quá trình quét và xây dựng bản đồ trước đó. Đây là dạng bản đồ occupancy grid, trong đó vùng sáng biểu thị khu vực có thể di chuyển, còn vùng tối

là tường hoặc vật cản. Bản đồ này được sử dụng trong các thuật toán định vị và lập kế hoạch đường đi, giúp robot xác định vị trí và di chuyển chính xác trong môi trường.



2. Các topic ROS2 cần chú ý trong hệ thống

Trong hệ thống ROS2 Navigation (Nav2), các topic đóng vai trò trao đổi dữ liệu giữa các node như định vị, bản đồ, cảm biến và điều khiển robot. Dưới đây là các topic cần chú ý được sử dụng trong hệ thống.

2.1. /map

- Vai trò: Cung cấp bản đồ môi trường tĩnh (Occupancy Grid)
- Publisher: map_server
- Subscriber: amcl, global_costmap, rviz2
- Ý nghĩa:
 - map_server phát bản đồ cho toàn hệ thống
 - AMCL dùng để định vị robot trên bản đồ
 - Costmap dùng để lập kế hoạch đường đi
 - RViz dùng để hiển thị

2.2. /odom

- Vai trò: Cung cấp trạng thái chuyển động (vị trí + vận tốc robot)
- Publisher: (robot driver / ekf_filter_node)
- Subscriber: bt_navigator, navigation stack
- Ý nghĩa:
 - Cung cấp thông tin di chuyển liên tục của robot
 - Dùng để ước lượng quãng đường và hỗ trợ điều hướng

2.3. /amcl_pose

- Vai trò: Cung cấp vị trí robot trên bản đồ (localization)
- Publisher: amcl
- Subscriber: bt_navigator, RViz, costmap
- Ý nghĩa:
 - AMCL tính toán vị trí chính xác của robot
 - Là dữ liệu quan trọng nhất để robot biết “mình đang ở đâu”

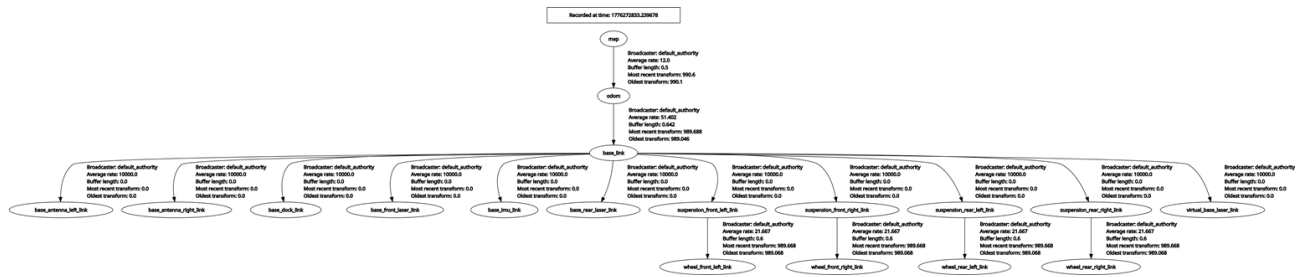
2.4. /scan_front_raw

- Vai trò: Dữ liệu LiDAR phía trước robot
- Publisher: ros_gz_bridge (hoặc driver LiDAR)
- Subscriber: amcl, global_costmap, local_costmap, rviz2
- Ý nghĩa:
 - Cung cấp dữ liệu vật cản phía trước
 - AMCL dùng để hỗ trợ định vị
 - Costmap dùng để tránh va chạm
 - RViz dùng để hiển thị môi trường

2.5. /scan_rear_raw

- Vai trò: Dữ liệu LiDAR phía sau robot
- Publisher: ros_gz_bridge / sensor driver
- Subscriber: local_costmap, navigation stack
- Ý nghĩa:
 - Hỗ trợ phát hiện vật cản phía sau
 - Quan trọng khi robot lùi hoặc quay đầu

2.6. /tf



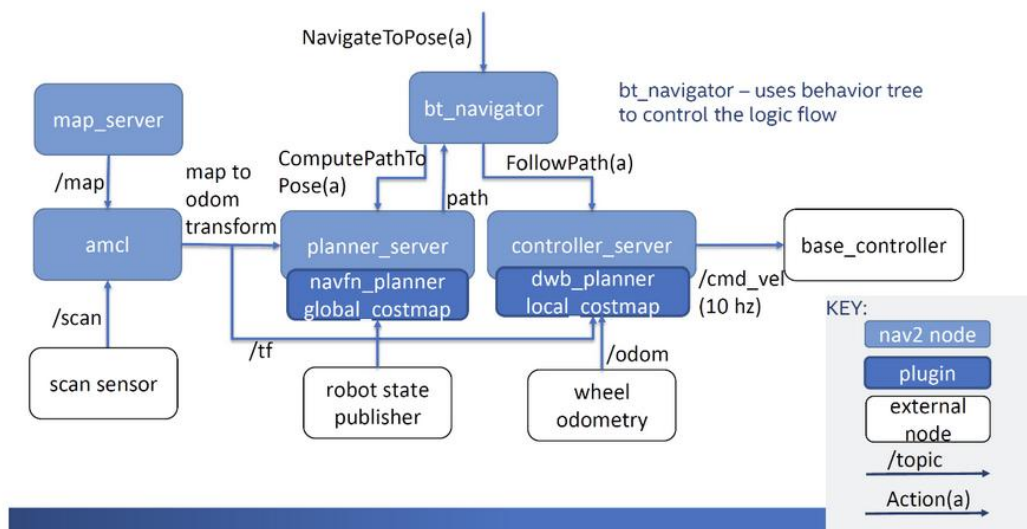
- Vai trò: Hệ thống biến đổi tọa độ giữa các frame
- Publisher: robot_state_publisher, amcl, ekf_filter_node
- Subscriber: hầu hết các node (AMCL, costmap, planner, controller, RViz)
- Ý nghĩa:
 - Kết nối các hệ tọa độ: map $\rightarrow$ odom $\rightarrow$ base_link $\rightarrow$...
 - Là nền tảng của toàn bộ hệ thống ROS2 Navigation
 - Sai TF $\rightarrow$ toàn bộ robot định vị sai

2.7. /mobile_base_controller/reference

- Vai trò: Lệnh điều khiển chuyển động robot
- Publisher: controller_server (Nav2)
- Subscriber: robot base driver
- Ý nghĩa:
 - Nhận lệnh tốc độ từ hệ thống navigation
 - Điều khiển robot di chuyển thực tế

3. Các server trong hệ thống launch Nav2

Nav2 achitecture



14

Trong hệ thống ROS2 Navigation (Nav2), các server đóng vai trò xử lý chính các chức năng như định vị, lập kế hoạch và điều khiển robot. Dưới đây là các server thuộc kiến trúc Nav2 trong launch file.

3.1. Map Server (nav2_map_server)

- Tên: `map_server`
- Chức năng:
 - Cung cấp và publish bản đồ môi trường (Occupancy Grid)
- Vai trò:
 - Là dữ liệu nền cho toàn bộ hệ thống Nav2
 - Được AMCL và global costmap sử dụng

3.2. AMCL (nav2_amcl)

- Tên: `amcl`
- Chức năng:
 - Định vị robot trên bản đồ bằng thuật toán Particle Filter
- Vai trò:
 - Xác định vị trí hiện tại của robot (pose)
 - Là thành phần chính của localization trong Nav2

3.3. Planner Server (nav2_planner)

- Tên: `planner_server`

- Chức năng:
 - Tính toán đường đi toàn cục từ vị trí hiện tại đến goal
- Vai trò:
 - Tạo global path dựa trên map và costmap
 - Đưa đường đi cho controller xử lý tiếp

3.4. Controller Server (nav2_controller)

- Tên: controller_server
- Chức năng:
 - Điều khiển robot di chuyển theo đường đã lập
- Vai trò:
 - Chuyển global path thành lệnh vận tốc (cmd_vel)
 - Điều khiển chuyển động thực tế của robot

3.5. Behavior Server / Recoveries Server (nav2_behaviors)

- Tên: behavior_server / recoveries_server
- Chức năng:
 - Xử lý các hành vi đặc biệt khi robot gặp lỗi
- Vai trò:
 - Recovery khi robot bị kẹt
 - Thực hiện các hành vi như quay tại chỗ, lùi, thoát vùng kẹt

3.6. BT Navigator (nav2_bt_navigator)

- Tên: bt_navigator
- Chức năng:
 - Điều phối toàn bộ quá trình navigation bằng Behavior Tree
- Vai trò:
 - Quản lý luồng hoạt động:
 - nhận goal
 - lập kế hoạch
 - điều khiển robot
 - giám sát trạng thái

3.7. Lifecycle Manager

- Tên: lifecycle_manager_navigation
- Chức năng:
 - Quản lý vòng đời (lifecycle) của các Nav2 server
- Vai trò:
 - Khởi động và đồng bộ các server

- Đảm bảo hệ thống chạy đúng thứ tự và ổn định

4. Thuật toán tối ưu đường đi

Trong hệ thống, thuật toán được sử dụng để tối ưu thứ tự di chuyển qua các điểm nhằm giảm tổng quãng đường di chuyển của robot.

4.1. Ý tưởng chính

- Sử dụng bài toán *TSP (Travelling Salesman Problem)* để tìm thứ tự điểm tối ưu
- Kết hợp với hàm `nav.getPath()` của Nav2 để tính khoảng cách thực tế trên bản đồ

4.2. Cách tính chi phí

- Khoảng cách giữa hai điểm không dùng khoảng cách Euclid
- Mà dùng độ dài đường đi thực tế do Nav2 planner tạo ra

Nếu không có đường đi hợp lệ → gán chi phí là vô cùng (∞)

4.3. Thuật toán tối ưu

- Duyệt tất cả hoán vị của các điểm cần đi
- Tính tổng chi phí của từng lộ trình
- Chọn lộ trình có tổng chi phí nhỏ nhất

4.4. Điểm đặc biệt

- Có điểm trung gian (pre-point) cho một số vị trí để đảm bảo an toàn khi tiếp cận mục tiêu
- Có offset tọa độ để phù hợp với hệ tọa độ thực tế của robot

4.5. Kết luận

Thuật toán giúp robot:

- Tối ưu thứ tự di chuyển
- Giảm quãng đường đi
- Tăng hiệu quả di chuyển trong môi trường thực tế